

EventFlow

Spring Annotations Explained

Every Spring annotation used in this project — what it does, where it appears, and what effect it has on the running application.

Files covered in this guide:

<code>EventFlowApplication.java</code>	Entry point
<code>config/AppConfig.java</code>	Bean definitions
<code>config/EventFlowProperties.java</code> + <code>AnthropicProperties.java</code>	Configuration binding
<code>config/DataInitializer.java</code>	Startup data seeding
<code>model/User.java</code> , <code>Event.java</code> , <code>Registration.java</code> , <code>AiConversation.java</code>	JPA entities
<code>repository/*.java</code>	Spring Data repositories
<code>service/*.java</code>	Business logic services
<code>controller/*.java</code>	MVC controllers
<code>aspect/LoggingAspect.java</code>	AOP aspect
<code>security/SecurityConfig.java</code> + <code>SecurityUtils.java</code>	Spring Security
<code>dto/EventFlowDtos.java</code>	Validation DTOs
<code>application.yaml</code>	Configuration properties

1. Application Entry Point

EventFlowApplication.java

■ EventFlowApplication.java

@SpringBootApplication

The single most important annotation in the whole project. It is a shortcut for three annotations in one: **@SpringBootConfiguration** (marks this as a config source), **@EnableAutoConfiguration** (tells Spring to read the classpath and auto-configure matching beans), and **@ComponentScan** (tells Spring to scan the *com.eventflow* package and every sub-package for classes annotated with **@Component**, **@Service**, **@Repository**, **@Controller**, etc.).

```
@SpringBootApplication // three annotations in one
@ConfigurationPropertiesScan // also scan for @ConfigurationProperties classes
@EnableAsync // allow @Async on methods (run in background threads)
public class EventFlowApplication {
    public static void main(String[] args) {
        SpringApplication.run(EventFlowApplication.class, args);
    }
}
```

■ Console on startup:
■ Admin created – email: admin@eventflow.com password: Admin@1234
■ EventFlow data initialization complete

■ Note

Because **@ComponentScan** is included, you never need to list every class manually. Spring finds *UserService*, *EventService*, *SecurityConfig*, etc. automatically just because they are in sub-packages.

@ConfigurationPropertiesScan

Without this annotation, classes annotated with **@ConfigurationProperties** would need to be manually registered. This scan automatically discovers *EventFlowProperties* and *AnthropicProperties* and registers them as beans whose fields are bound from *application.yaml*.

@EnableAsync

Enables Spring's asynchronous method execution support. Any bean method annotated with **@Async** will run in a separate thread pool instead of blocking the HTTP request thread. In EventFlow this is declared at the application level so that any future async operations (e.g. sending emails after registration) can be added just by annotating a service method with **@Async**.

2. Configuration Properties Binding

config/EventFlowProperties.java · config/AnthropicProperties.java · application.yaml

■ **config/EventFlowProperties.java & config/AnthropicProperties.java**

@ConfigurationProperties("prefix")

Binds a whole group of YAML/properties values to a Java class. Spring reads all keys that start with the given prefix and maps them to fields with matching names (using relaxed binding: *default-password* in YAML → *defaultPassword* in Java).

```
// EventFlowProperties.java
@ConfigurationProperties("eventflow") // binds everything under 'eventflow:' in yaml
@Data // Lombok: generates getters/setters automatically
public class EventFlowProperties {
    private Ai ai = new Ai();
    private Admin admin = new Admin();
    private Pagination pagination = new Pagination();

    @Data
    public static class Admin {
        private String defaultPassword = "Admin@1234"; // fallback default
    }
}
```

The matching section in *application.yaml*:

```
# application.yaml
eventflow:
  admin:
    default-password: Admin@1234 # → eventFlowProperties.getAdmin().getDefaultPassword()
  ai:
    enabled: true
    system-prompt: "You are EventFlow AI..."
  pagination:
    default-page-size: 12
■ DataInitializer uses it: props.getAdmin().getDefaultPassword() → "Admin@1234"
```

AnthropicProperties — same pattern for the AI API:

```
@ConfigurationProperties("anthropic.api") // prefix = 'anthropic.api'
@Data
public class AnthropicProperties {
    private String key; // anthropic.api.key
    private String baseUrl = "https://api.anthropic.com";
    private String model = "claude-opus-4-20250514";
    private int maxTokens = 2048;
}
■ anthropicProperties.getModel() → "claude-opus-4-20250514"
anthropicProperties.getKey() → value of ANTHROPIC_API_KEY env var
```

■ Note

The YAML value `key: ${ANTHROPIC_API_KEY:your-api-key-here}` means: read from the environment variable `ANTHROPIC_API_KEY`; if it is not set, use the default 'your-api-key-here'. This is the standard way to keep secrets out of source code.

@Data (Lombok)

Not a Spring annotation but used everywhere. Lombok's `@Data` generates: getters for all fields, setters for all non-final fields, `equals()`, `hashCode()`, and `toString()` — all at compile time. It keeps the configuration classes clean

with no boilerplate.

3. @Configuration & @Bean

config/AppConfig.java

■ config/AppConfig.java

@Configuration

Marks a class as a *configuration class* — a place where you define beans using `@Bean` methods. Spring processes these classes specially: when you call a `@Bean` method from within another `@Bean` method in the same class, Spring intercepts the call and returns the existing singleton bean instead of creating a new object.

@Bean

Declares that a method's return value should be registered as a bean in the Spring IoC container. Use `@Bean` when you cannot add `@Component` to the class (e.g. third-party classes like `WebClient` or `ObjectMapper`), or when construction requires complex logic.

```
@Configuration
@RequiredArgsConstructor // Lombok: generates constructor for all final fields
public class AppConfig {

    private final AnthropicProperties anthropicProperties; // injected by Spring

    @Bean // this WebClient instance is created once and shared everywhere
    public WebClient anthropicWebClient() {
        return WebClient.builder()
            .baseUrl(anthropicProperties.getBaseUrl()) // https://api.anthropic.com
            .defaultHeader("x-api-key", anthropicProperties.getKey())
            .defaultHeader("anthropic-version", "2023-06-01")
            .defaultHeader("content-type", "application/json")
            .codecs(c -> c.defaultCodecs().maxInMemorySize(2 * 1024 * 1024))
            .build();
    }

    @Bean // global ObjectMapper: handles Java 8 dates (LocalDate, LocalDateTime)
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule());
        mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        return mapper; // LocalDate serialized as "2025-04-10" not 1744329600000
    }
}

■ AnthropicService receives WebClient via constructor injection.
ObjectMapper serializes LocalDate as "2025-04-10" in JSON responses.
```

■ Note

Because `@RequiredArgsConstructor` is used, Spring automatically injects `AnthropicProperties` via the constructor. No `@Autowired` needed when there is exactly one constructor.

4. Startup Data Seeding

config/DataInitializer.java

■ config/DataInitializer.java

CommandLineRunner interface

Not an annotation but a Spring interface. Any bean that implements CommandLineRunner has its run() method called automatically by Spring Boot after the application context is fully initialized and the embedded server is ready. It is the standard pattern for seeding initial data.

@Component

The generic stereotype annotation that tells Spring: 'create a bean from this class and manage it.' DataInitializer is marked @Component (not @Service) because it is infrastructure code, not business logic.

@Transactional on run()

Wraps the entire run() method in a single database transaction. If any of the seed operations fail (e.g., saving an event throws an exception), the whole initialization is rolled back, leaving the database clean.

```
@Component
@RequiredArgsConstructor
@Slf4j // Lombok: injects 'log' field (SLF4J logger)
public class DataInitializer implements CommandLineRunner {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final EventFlowProperties props;

    @Override
    @Transactional // all DB writes in one transaction – rollback if any fails
    public void run(String... args) {
        seedAdmin(); // creates admin@eventflow.com if not exists
        seedDataAdder(); // creates manager@eventflow.com if not exists
        seedSampleEvents(); // creates 6 sample events if no events exist
        log.info("■ EventFlow data initialization complete");
    }

    private void seedAdmin() {
        if (!userRepository.existsByEmail("admin@eventflow.com")) {
            User admin = new User("Admin", "EventFlow",
                "admin@eventflow.com",
                passwordEncoder.encode(props.getAdmin().getDefaultPassword()),
                User.Role.ADMIN);
            userRepository.save(admin);
        }
    }

    private void seedDataAdder() {
        if (!userRepository.existsByEmail("manager@eventflow.com")) {
            User manager = new User("Manager", "EventFlow",
                "manager@eventflow.com",
                passwordEncoder.encode(props.getManager().getDefaultPassword()),
                User.Role.MANAGER);
            userRepository.save(manager);
        }
    }

    private void seedSampleEvents() {
        if (props.getSampleEvents().size() == 0) {
            List<Event> events = new ArrayList<>();
            for (int i = 0; i < 6; i++) {
                Event event = new Event();
                event.setTitle("Sample Event " + (i + 1));
                event.setDescription("Sample Event " + (i + 1) + " description");
                event.setCategory("Sample Event " + (i + 1));
                event.setStartAt(new Date());
                event.setEndAt(new Date());
                event.setCreatedBy("admin@eventflow.com");
                event.setUpdatedBy("admin@eventflow.com");
                event.setCreatedAt(new Date());
                event.setUpdatedAt(new Date());
                events.add(event);
            }
            eventRepository.saveAll(events);
        }
    }
}
```

■ Startup console output:

```
■ Admin created – email: admin@eventflow.com password: Admin@1234
■ Sample DATA_ADDER created – email: manager@eventflow.com
■ 6 sample events created
■ EventFlow data initialization complete
```

■ Note

The check `if (!userRepository.existsByEmail(...))` makes the seeder **idempotent** — safe to run on every startup without duplicating data. This is critical because `ddl-auto: create-drop` recreates the schema on each restart.

5. JPA Entity Annotations

model/User.java · Event.java · Registration.java · AiConversation.java

■ [model/User.java](#) | [model/Event.java](#) | [model/Registration.java](#) | [model/AiConversation.java](#)

@Entity and @Table

@Entity tells JPA (Hibernate) that this class maps to a database table. **@Table(name = "users")** specifies the exact table name. Without **@Table**, Hibernate would use the class name as the table name.

```
@Entity
@Table(name = "users") // → creates/uses table called 'users' in the DB
@Getter @Setter // Lombok: generate getters + setters for all fields
@NoArgsConstructor // Lombok: generate no-arg constructor (REQUIRED by JPA)
@ToString(exclude = {"registrations"}) // Lombok: exclude collection to avoid lazy-loading
public class User { ... }

@Entity
@Table(name = "registrations",
uniqueConstraints = @UniqueConstraint(columnNames = {"user_id", "event_id"}))
// ↑ database-level constraint: a user can only register for the same event once
public class Registration { ... }
■ Hibernate generates SQL:
CREATE TABLE users (id BIGINT AUTO_INCREMENT PRIMARY KEY, email VARCHAR(150) UNIQUE, ...)
CREATE TABLE registrations (... UNIQUE(user_id, event_id))
```

@Id and @GeneratedValue

@Id marks the primary key field. **@GeneratedValue(strategy = GenerationType.IDENTITY)** delegates key generation to the database (AUTO_INCREMENT in H2/MySQL, SERIAL in PostgreSQL). You never set the id manually — the DB assigns it on INSERT.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
// When you call eventRepository.save(new Event(...)):
// → Hibernate runs: INSERT INTO events (...) VALUES (...)
// → DB assigns id=1 (or next auto-increment value)
// → Hibernate sets event.id = 1 on the returned object
■ event.getId() → null before save
eventRepository.save(event); → event.getId() → 1
```

@Column customizations

Customizes how a field maps to a database column. Without **@Column**, Hibernate uses the field name as the column name and allows nulls.

```
@Column(nullable = false, length = 200) // NOT NULL VARCHAR(200)
private String title;

@Column(nullable = false, columnDefinition = "TEXT") // unlimited-length text column
private String description;

@Column(nullable = false, unique = true, length = 150) // UNIQUE constraint
private String email;

@Column(uptodate = false) // set once at insert, never updated
private LocalDateTime createdAt;
```

@Enumerated(EnumType.STRING)

Tells JPA to store enum values as their string name (e.g. 'ACTIVE', 'TECH') rather than as integers (0, 1, 2).

Always use STRING — ORDINAL breaks if you ever add/reorder enum values.

```
@Enumerated(EnumType.STRING)
@Column(nullable = false)
private Category category = Category.OTHER;
// DB stores: 'TECH' or 'MUSIC' or 'WELLNESS' etc.
// NOT 0, 1, 2 (that would be EnumType.ORDINAL – fragile)
■ DB column: category VARCHAR → value: 'TECH'
Query: SELECT * FROM events WHERE category = 'TECH'
```

@ManyToOne and @OneToMany

Define the relationship between entities. The event has many registrations (OneToMany); each registration belongs to one event (ManyToOne). **mappedBy** tells JPA which side owns the foreign key column.

CascadeType.ALL means: when you delete a User, all their Registrations are also deleted.

```
// In Event.java – one event has many registrations
@OneToMany(
    mappedBy = "event", // 'event' field in Registration owns the FK
    cascade = CascadeType.ALL, // delete event → delete all its registrations
    fetch = FetchType.LAZY // registrations NOT loaded until accessed
)
private Set<Registration> registrations = new HashSet<>();

// In Registration.java – each registration belongs to one event
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "event_id", nullable = false) // FK column in 'registrations' table
private Event event;
■ DB: registrations.event_id = 5 → refers to events.id = 5
Deleting event id=5 → also deletes all registrations where event_id=5
```

@CreationTimestamp and @UpdateTimestamp (Hibernate)

Hibernate automatically sets these fields. **@CreationTimestamp** sets the field to the current timestamp when the entity is first persisted (INSERT). **@UpdateTimestamp** updates the field every time the entity is saved (INSERT or UPDATE). Both are read-only from the application's perspective.

```
@CreationTimestamp
@Column(updatable = false) // never changed after first insert
private LocalDateTime createdAt;

@UpdateTimestamp
private LocalDateTime updatedAt; // changes on every save()
■ After userRepository.save(newUser):
createdAt → 2025-04-10T14:30:00
updatedAt → 2025-04-10T14:30:00

After userRepository.save(existingUser):
updatedAt → 2025-04-10T15:45:22 (createdAt unchanged)
```

■ Watch
out

Using **FetchType.LAZY** on @ManyToOne/@OneToMany means the related data is NOT loaded from the DB until you access it. If you access it after the transaction has closed, you get a **LazyInitializationException**. The fix is **@Transactional** on the service method.

6. Bean Validation Annotations

dto/EventFlowDtos.java · model/*.java

■ dto/EventFlowDtos.java

Validation annotations (from *jakarta.validation*) define rules on fields. In Spring MVC they are activated by **@Valid** on a controller method parameter. If validation fails, Spring populates a **BindingResult** with errors and the controller can return the form view again with error messages.

Annotation	What it validates	Used on in EventFlow
@NotBlank	String is not null, not empty, not whitespace-only	title, email, password, firstName
@NotNull	Value is not null	eventDate, eventTime
@Size(min=X, max=Y)	String length between min and max	title (3–200), description (10–5000)
@Email	Valid email address format	email fields in User and DTOs
@Min(X) / @Max(Y)	Numeric value at least X / at most Y	maxParticipants (1–100000)
@DecimalMin("0.0")	Decimal ≥ 0.0	price field in Event and DTO
@FutureOrPresent	Date is today or in the future	eventDate in EventCreateDto
@UniqueConstraint	Database-level unique constraint (on @Table)	Registration: user_id + event_id pair

```
// In EventFlowDtos.java – the form input DTO
@Data
@NoArgsConstructor
public static class EventCreateDto {
    @NotBlank(message = "Title is required")
    @Size(min = 3, max = 200)
    private String title;

    @NotNull(message = "Date is required")
    @FutureOrPresent(message = "Event date must be today or in the future")
    private LocalDate eventDate;

    @Min(value = 1, message = "Must allow at least 1 participant")
    @Max(value = 100000)
    private int maxParticipants;
}

// In the controller – @Valid activates validation
@PostMapping("/events/new")
public String createEvent(
    @Valid @ModelAttribute("dto") EventCreateDto dto,
    BindingResult result, // holds any validation errors
    RedirectAttributes ra) {
    if (result.hasErrors()) {
        return "manage/event-form"; // back to form, Thymeleaf shows errors
    }
    // only reached if all @Valid constraints pass
    eventService.createEvent(dto, user);
    ra.addFlashAttribute("success", "Event created!");
    return "redirect:/manage/events";
}

■ If title is blank → result.hasErrors() = true → form re-shown with:
'Title is required' displayed next to the title field
```

```
If date is in the past → 'Event date must be today or in the future'
```

7. Spring Data JPA Repository Annotations

repository/*.java

■ repository/EventRepository.java | UserRepository.java | RegistrationRepository.java | AiConversationRepository.java

@Repository and JpaRepository

@Repository marks the interface as a Spring-managed data access component. Extending **JpaRepository<Entity, ID>** gives you ~20 free CRUD methods. Spring generates the implementation at runtime — you write zero SQL for basic operations.

```
@Repository
public interface EventRepository extends JpaRepository<Event, Long> {
    // FREE methods from JpaRepository:
    // save(event) → INSERT or UPDATE
    // findById(id) → SELECT by PK → Optional<Event>
    // findAll() → SELECT all events
    // delete(event) → DELETE
    // count() → SELECT COUNT(*)
    // existsById(id) → true/false
}
```

Derived Query Methods — SQL from the method name

Spring Data reads the method name and builds the SQL query automatically. No implementation needed — just the interface declaration.

```
// EventRepository
Page<Event> findByStatusOrderByEventDateAsc(Event.Status status, Pageable pageable);
// SQL: SELECT * FROM events WHERE status = ? ORDER BY event_date ASC (paginated)

Page<Event> findByCategoryAndStatusOrderByEventDateAsc(
    Event.Category category, Event.Status status, Pageable pageable);
// SQL: SELECT * FROM events WHERE category = ? AND status = ? ORDER BY event_date ASC

// UserRepository
Optional<User> findByEmail(String email);
// SQL: SELECT * FROM users WHERE email = ? (returns Optional – empty if not found)

boolean existsByEmail(String email);
// SQL: SELECT EXISTS(SELECT 1 FROM users WHERE email = ?)

long countByRole(User.Role role);
// SQL: SELECT COUNT(*) FROM users WHERE role = ?

// RegistrationRepository
boolean existsByUserAndEvent(User user, Event event);
// SQL: SELECT EXISTS(... WHERE user_id = ? AND event_id = ?)
// Used to prevent duplicate registrations

long countByEventAndStatus(Event event, Registration.RegistrationStatus status);
// SQL: SELECT COUNT(*) FROM registrations WHERE event_id = ? AND status = ?
// Used to count confirmed attendees

// AiConversationRepository
List<AiConversation> findTop20ByUserOrderByCreatedAtDesc(User user);
// SQL: SELECT * FROM ai_conversations WHERE user_id = ?
// ORDER BY created_at DESC LIMIT 20
// Returns the 20 most recent AI chats for this user
```

```

■ findByEmail("john@example.com") → Optional[User{id=3, email='john@example.com', ...}]
countByEventAndStatus(event, CONFIRMED) → 12
findTop20ByUser(user) → [AiConversation{...}, AiConversation{...}, ...] (max 20)

```

@Query — Custom JPQL queries

When the method name syntax is not enough, write JPQL directly. JPQL uses *entity names and field names*, not table/column names. **:param** syntax (named parameters) is more readable than **?1** positional parameters.

```

// Full-text search across title, description, and location
@Query("SELECT e FROM Event e WHERE e.status = 'ACTIVE' AND (" +
"LOWER(e.title) LIKE LOWER(CONCAT('%', :query, '%')) OR " +
"LOWER(e.description) LIKE LOWER(CONCAT('%', :query, '%')) OR " +
"LOWER(e.location) LIKE LOWER(CONCAT('%', :query, '%'))")
Page<Event> searchEvents(@Param("query") String query, Pageable pageable);
// Called: eventRepository.searchEvents("jazz", PageRequest.of(0, 12))
// SQL: WHERE status='ACTIVE' AND (LOWER(title) LIKE '%jazz%' OR ...)

// Upcoming events from today onwards
@Query("SELECT e FROM Event e WHERE e.status = 'ACTIVE' AND e.eventDate >= :fromDate ORDER BY
e.eventDate ASC")
Page<Event> findUpcomingEvents(@Param("fromDate") LocalDate fromDate, Pageable pageable);

// Category breakdown for the admin dashboard chart
@Query("SELECT e.category, COUNT(e) FROM Event e GROUP BY e.category ORDER BY COUNT(e) DESC")
List<Object[]> countByCategory();
// Returns: [["TECH", 3], ["MUSIC", 2], ["COMMUNITY", 1], ...]

// JOIN FETCH: load registrations+events together to avoid N+1 queries
@Query("SELECT r FROM Registration r JOIN FETCH r.event WHERE r.user = :user AND r.status =
'CONFIRMED' ORDER BY r.event.eventDate ASC")
List<Registration> findConfirmedWithEvents(@Param("user") User user);
■ searchEvents("jazz") → [Event{title="Wrocław Jazz Night", ...}]
countByCategory() → [[TECH,3],[MUSIC,1],[COMMUNITY,1]]

```

■ Note

JOIN FETCH in the last query solves the N+1 problem: without it, accessing `registration.getEvent()` for each of 20 registrations would fire 20 separate SQL queries. JOIN FETCH loads everything in one query.

8. Service Layer Annotations

service/*.java

■ service/EventService.java | UserService.java | RegistrationService.java | AnthropicService.java | AiCommandService.java

@Service

Marks a class as a business logic component. Functionally equivalent to `@Component` but semantically communicates that this class implements use cases. Spring creates one singleton instance of each `@Service` class and injects it wherever it is needed.

@Transactional — the most important service annotation

Spring wraps the method in a database transaction. If the method completes normally → Spring commits. If a `RuntimeException` is thrown → Spring rolls back. For read-only queries, `@Transactional(readOnly = true)` gives a performance hint to Hibernate (no dirty checking, faster flush).

```
@Service
@RequiredArgsConstructor
@Slf4j
public class RegistrationService {

    @Transactional // single transaction: check + save are atomic
    public Registration registerForEvent(User user, Long eventId) {
        Event event = eventRepository.findById(eventId).orElseThrow(...);

        // all checks in the same transaction – no race conditions
        if (event.getStatus() != Event.Status.ACTIVE)
            throw new IllegalStateException("Event not accepting registrations");
        if (event.isPast())
            throw new IllegalStateException("Cannot register for past event");
        if (registrationRepository.existsByUserAndEvent(user, event))
            throw new IllegalStateException("Already registered");
        if (event.isFull())
            throw new IllegalStateException("Event is full");

        Registration reg = new Registration(user, event);
        reg.setStatus(Registration.RegistrationStatus.CONFIRMED);
        return registrationRepository.save(reg); // INSERT
        // ↑ if any exception happened above, this line is never reached
        // and the whole transaction is rolled back
    }

    @Transactional(readOnly = true) // no dirty checking, faster
    public List<Registration> getUserRegistrations(User user) {
        return registrationRepository.findConfirmedWithEvents(user);
    }
}

■ registerForEvent(user, eventId=5):
→ If event is full: throws IllegalStateException → transaction rolled back
→ If ok: INSERT into registrations → committed
→ log: "User john@example.com registered for event 'Jazz Night'"
```

@Slf4j (Lombok) — logging in every service

Lombok's `@Slf4j` injects a private static final `Logger log` field. You use it as `log.info()`, `log.warn()`, `log.error()`. Combined with the YAML setting `logging.level.com.eventflow: DEBUG`,

every log statement is visible in the console during development.

```
@Slf4j
public class EventService {
    // Lombok injects: private static final Logger log = LoggerFactory.getLogger(EventService.class);

    @Transactional
    public Event createEvent(EventCreateDto dto, User createdBy) {
        Event saved = eventRepository.save(event);
        log.info("Event created: '{}{}' by {}", saved.getTitle(), createdBy.getEmail());
        // ↑ outputs: INFO c.e.service.EventService - Event created: 'Jazz Night' by admin@eventflow.com
        return saved;
    }

    @Transactional
    public void deleteEvent(Long id, User requestingUser) {
        // ...
        log.warn("Event '{}{}' deleted by {}", event.getTitle(), requestingUser.getEmail());
        // ↑ WARN level – deletions are warnings
    }
}

■ Console: INFO c.e.service.EventService - Event created: 'Jazz Night' by manager@eventflow.com
```

9. MVC Controller Annotations

controller/*.java

■ controller/PublicController.java | AuthController.java | ManageController.java | AdminController.java

@Controller

Marks the class as a Spring MVC controller. Methods return view names (Thymeleaf template paths) or the string 'redirect:/path'. Unlike @RestController, it does NOT assume every method returns JSON — it returns view names for Thymeleaf rendering.

@RequestMapping at class level

Defines a *base path prefix* for all methods in that controller. Every method's mapping is relative to this prefix.

```
@Controller
@RequestMapping("/admin") // ALL methods in this class are under /admin/...
@PreAuthorize("hasRole('ADMIN')") // ALL methods require ADMIN role
@RequiredArgsConstructor
public class AdminController {

    @GetMapping("/dashboard") // → handles GET /admin/dashboard
    public String dashboard(Model model) {
        model.addAttribute("stats", ...);
        return "admin/dashboard"; // → renders templates/admin/dashboard.html
    }

    @GetMapping("/users") // → handles GET /admin/users
    public String userList(...) { return "admin/users"; }

    @PostMapping("/users/{id}/promote") // → handles POST /admin/users/5/promote
    public String promoteUser(@PathVariable Long id, RedirectAttributes ra) {
        userService.promoteToDataAdder(id, admin);
        ra.addFlashAttribute("success", "User promoted!");
        return "redirect:/admin/users"; // browser redirects to /admin/users
    }
}

■ GET /admin/dashboard → renders admin/dashboard.html with stats
POST /admin/users/5/promote → promotes user 5, redirects to /admin/users
Flash message 'User promoted!' shown on the /admin/users page
```

@GetMapping, @PostMapping

Map specific HTTP methods to controller methods. **GET** is for loading pages and data. **POST** is for form submissions and mutations.

@PathVariable

Extracts a value from the URL path. In /events/{id}, the {id} placeholder becomes the parameter.

```
@GetMapping("/events/{id}")
public String eventDetail(@PathVariable Long id, Model model) {
    Event event = eventService.findByIdOrThrow(id); // id = 42 from URL /events/42
    model.addAttribute("event", event);
    return "events/detail";
}

■ GET /events/42 → event = Event{id=42, title="Jazz Night"} → renders events/detail.html
```

@RequestParam

Reads a query string parameter from the URL. `defaultValue` makes it optional.

```
@GetMapping("/events")
public String eventsList(
    @RequestParam(defaultValue = "0") int page, // /events?page=2 → page=2 (default: 0)
    @RequestParam(defaultValue = "") String category, // /events?category=TECH
    @RequestParam(defaultValue = "") String q, // /events?q=jazz (search)
    Model model) {

    Page<Event> events;
    if (!q.isBlank()) {
        events = eventService.searchEvents(q, page, 12);
    } else if (!category.isBlank()) {
        events = eventService.findByCategory(Event.Category.valueOf(category), page, 12);
    } else {
        events = eventService.findUpcoming(page, 12);
    }
    model.addAttribute("events", events);
    return "events/list";
}

■ GET /events?q=jazz → searchEvents("jazz", 0, 12) → [Jazz Night, ...]
GET /events?category=TECH&page=1 → findByCategory(TECH, 1, 12)
```

@ModelAttribute

Binds HTML form fields to a Java DTO object. Spring reads all form field names, finds matching setter/field names in the DTO class, and sets the values. Combined with `@Valid` it also runs Bean Validation.

```
@PostMapping("/register")
public String register(
    @Valid @ModelAttribute("dto") UserRegistrationDto dto, // form → DTO
    BindingResult result,
    RedirectAttributes redirectAttributes) {

    if (!dto.passwordsMatch()) {
        result.rejectValue("confirmPassword", "mismatch", "Passwords do not match");
    }
    if (result.hasErrors()) {
        return "auth/register"; // re-show form with errors
    }
    userService.registerUser(dto);
    redirectAttributes.addFlashAttribute("successMessage", "Account created!");
    return "redirect:/auth/login";
}

■ POST /auth/register with form data firstName=John&lastName=Doe&email=john@mail.com&password=Abc123!!
→ dto.getFirstName() = "John"
→ If passwords don't match: form re-shown with error under confirmPassword field
```

@ResponseBody

Makes the method return its value directly to the HTTP response body as JSON (via Jackson), bypassing Thymeleaf view resolution. Used in `ManageController` for the AI chat API endpoint.

```
// In ManageController – REST-style endpoint inside a @Controller
@PostMapping("/ai/chat")
@ResponseBody // returns JSON, not a view name
public ResponseEntity<AiChatResponseDto> chat(@RequestBody AiChatRequestDto request) {
    User user = securityUtils.getCurrentUserOrThrow();
    AiChatResponseDto response = aiCommandService.handleMessage(request.getMessage(), user);
    return ResponseEntity.ok(response); // HTTP 200 + JSON body
}
```

```
■ POST /manage/ai/chat Body: {"message": "create a tech event tomorrow"}
→ HTTP 200
→ Body: {"action":"CREATE","message":"Created 'Tech Event'...", "success":true}
```

RedirectAttributes and Flash Messages

RedirectAttributes.addFlashAttribute() stores a value in the HTTP session for *exactly one* redirect. After the redirect, the value is retrieved from the session and added to the model automatically by Spring. This is how success/error messages survive the redirect.

```
// After deleting an event:
ra.addFlashAttribute("success", "Event deleted."); // stored in session
return "redirect:/manage/events"; // browser redirects

// On the /manage/events page – Thymeleaf reads the flash attribute:
// th:if="${success}" → displays "Event deleted." once, then it's gone
■ User clicks Delete → POST /manage/events/5/delete
→ Spring stores flash message
→ Browser redirects to /manage/events
→ Flash 'Event deleted.' shown once at top of page
```

10. Spring Security Annotations

security/SecurityConfig.java · security/SecurityUtils.java

■ security/SecurityConfig.java | security/SecurityUtils.java

@EnableWebSecurity

Activates Spring Security's web security support. Without this, Spring Security's auto-configuration provides a basic setup. With @EnableWebSecurity you take full control by providing a SecurityFilterChain bean.

@EnableMethodSecurity

Enables method-level security annotations like @PreAuthorize. Without this, @PreAuthorize on controller methods is silently ignored.

```
@Configuration
@EnableWebSecurity // take control of HTTP security
@EnableMethodSecurity // enable @PreAuthorize, @PostAuthorize on methods
@RequiredArgsConstructor
public class SecurityConfig {

    @Bean // defines URL-level security rules
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                // Public – no login required
                .requestMatchers("/", "/events", "/events/{id}", "/auth/**").permitAll()
                .requestMatchers("/css/**", "/js/**", "/img/**").permitAll()

                // Role-based access
                .requestMatchers("/admin/**").hasRole("ADMIN") // only ADMIN
                .requestMatchers("/manage/**").hasAnyRole("DATA_ADDER", "ADMIN")

                // Any authenticated user
                .requestMatchers("/register-event/**").authenticated()
                .anyRequest().authenticated() // everything else needs login
            )
            .formLogin(form -> form
                .loginPage("/auth/login") // custom login page
                .loginProcessingUrl("/auth/login") // Spring processes this POST
                .defaultSuccessUrl("/dashboard") // where to go after login
                .failureUrl("/auth/login?error=true")
            )
            .logout(logout -> logout
                .logoutRequestMatcher(new AntPathRequestMatcher("/auth/logout"))
                .logoutSuccessUrl("/")
                .invalidateHttpSession(true) // clears session data
                .clearAuthentication(true)
            )
            .rememberMe(rem -> rem
                .key("eventflow-remember-me-secret-key-2024")
                .tokenValiditySeconds(7 * 24 * 60 * 60) // 7 days
            );
        return http.build();
    }

    @Bean // BCrypt is the industry standard – stores hashed passwords
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

```
■ GET /admin/dashboard without login → redirect to /auth/login
GET /admin/dashboard as DATA_ADDER → 403 Forbidden
GET /events → allowed for everyone (public)
```

UserDetailsService Bean

This lambda is the bridge between your User entity and Spring Security. Spring Security calls this when processing a login form — it gives Spring the password hash to compare against and the role to set as an authority.

```
@Bean
public UserDetailsService userDetailsService() {
    return email -> { // Spring calls this with the submitted email
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " + email));
        if (!user.isActive()) {
            throw new DisabledException("Account is disabled"); // blocks deactivated users
        }
        return new org.springframework.security.core.userdetails.User(
            user.getEmail(),
            user.getPasswordHash(), // BCrypt hash – Spring compares automatically
            List.of(new SimpleGrantedAuthority("ROLE_" + user.getRole().name()))
            // user.getRole() = ADMIN → authority = "ROLE_ADMIN"
        );
    };
}

■ Login: email=admin@eventflow.com password=Admin@1234
→ userDetailsService loads User from DB
→ Spring verifies BCrypt(Admin@1234) matches stored hash
→ Sets security context with authority ROLE_ADMIN
→ Redirects to /dashboard → /admin/dashboard
```

@PreAuthorize

Secures a specific method with a Spring Expression Language (SpEL) expression. Spring evaluates the expression before the method runs. If false → AccessDeniedException (403).

```
// AdminController – locks entire controller to ADMIN only
@Controller
@RequestMapping("/admin")
@PreAuthorize("hasRole('ADMIN')") // checked on every method in this class
public class AdminController { ... }

// ManageController – accessible to DATA_ADDER or ADMIN
@Controller
@RequestMapping("/manage")
@PreAuthorize("hasAnyRole('DATA_ADDER', 'ADMIN')")
public class ManageController { ... }

■ DATA_ADDER visits /admin/dashboard → @PreAuthorize fails → 403
→ GlobalExceptionHandler.handleForbidden() → renders error/error.html
→ User sees 'You don't have permission to access this page.'
```

11. AOP — Aspect-Oriented Programming

aspect/LoggingAspect.java

■ `aspect/LoggingAspect.java`

AOP lets you inject behaviour around methods without modifying those methods. `LoggingAspect` automatically measures how long every service method takes, logs slow calls, and logs any exceptions — without a single line of logging code in the services themselves.

@Aspect

Tells Spring this class contains aspect definitions. Note: `@Aspect` alone does NOT make it a Spring bean. You must also add `@Component` (or `@Bean`) so Spring manages the instance.

@Component alongside @Aspect

Without `@Component`, Spring would never know about this class and the aspect would never run. This is a very common mistake — `@Aspect` marks *what* it is, `@Component` tells Spring to *manage* it.

@Pointcut

Defines a named, reusable pointcut expression. `within(com.eventflow.service..*)` means: any method called on any class inside the `com.eventflow.service` package or its sub-packages.

@Around

The most powerful advice type. It *wraps* the method call — runs before AND after. `ProceedingJoinPoint.proceed()` calls the actual method. You can inspect/modify arguments before calling it and inspect/modify the return value after.

```
@Aspect
@Component // BOTH needed: @Aspect = definition, @Component = Spring manages it
@Slf4j
public class LoggingAspect {

    @Pointcut("within(com.eventflow.service..*)") // matches ALL service methods
    public void serviceLayer() {} // empty method – just a name

    @Around("serviceLayer()") // intercepts every service method
    public Object logServiceCalls(ProceedingJoinPoint jp) throws Throwable {
        // jp.getSignature() = "EventService.createEvent"
        String method = jp.getSignature().getDeclaringType().getSimpleName()
            + "." + jp.getSignature().getName();

        long start = System.currentTimeMillis();
        try {
            Object result = jp.proceed(); // → calls the REAL service method
            long elapsed = System.currentTimeMillis() - start;

            if (elapsed > 500) {
                log.warn("■ SLOW SERVICE CALL: {} took {}ms", method, elapsed);
            } else {
                log.debug("→ {} completed in {}ms", method, elapsed);
            }
        } catch (Exception e) {
            log.error("X {} threw {}: {}", method, e.getClass().getSimpleName(), e.getMessage());
        }
        return result; // return whatever the real method returned
    }
}
```

```
throw e; // rethrow so the original exception still propagates
}
}
}
■ Normal call: DEBUG → EventService.createEvent completed in 12ms
Slow call: WARN ■ SLOW SERVICE CALL: AnthropicService.chat took 1450ms
Error: ERROR ✕ RegistrationService.registerForEvent threw IllegalStateException: Event is full
```

■ Note

The aspect intercepts ALL service methods automatically — you can add a new service and it gets logged/timed with zero extra code. This is the power of AOP: cross-cutting concerns (logging, timing, security checks) are defined once and applied everywhere.

■ Watch out

AOP proxies only work for Spring-managed beans. If you call `eventService.someMethod()` from *inside* `EventService`, the aspect does NOT fire — you're calling the real object, bypassing the proxy.

12. Global Exception Handling

controller/GlobalExceptionHandler.java

■ controller/GlobalExceptionHandler.java

@ControllerAdvice

Makes a class a global error handler that applies to ALL controllers in the application. Without it, unhandled exceptions show the default Spring error page (white label error). With @ControllerAdvice you render your own friendly error page.

@ExceptionHandler

Marks a method as the handler for a specific exception type. Spring routes any uncaught exception of that type (from any controller) to this method.

@ResponseStatus

Sets the HTTP response status code. Without it, even error handlers return 200 OK by default.

```
@ControllerAdvice // applies to ALL controllers globally
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalArgumentException.class) // catches bad input
    @ResponseStatus(HttpStatus.BAD_REQUEST) // → HTTP 400
    public String handleBadRequest(IllegalArgumentException ex, Model model) {
        model.addAttribute("errorCode", "400");
        model.addAttribute("errorMessage", ex.getMessage());
        return "error/error"; // renders templates/error/error.html
    }

    @ExceptionHandler(AccessDeniedException.class) // security blocks access
    @ResponseStatus(HttpStatus.FORBIDDEN) // → HTTP 403
    public String handleForbidden(Model model) {
        model.addAttribute("errorCode", "403");
        model.addAttribute("errorMessage", "You don't have permission.");
        return "error/error";
    }

    @ExceptionHandler(Exception.class) // catch-all fallback
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR) // → HTTP 500
    public String handleGeneral(Exception ex, Model model) {
        log.error("Unhandled exception", ex); // logs full stack trace
        model.addAttribute("errorCode", "500");
        model.addAttribute("errorMessage", "Something went wrong. Please try again.");
        return "error/error";
    }
}

■ GET /events/99999 (non-existent):
→ eventService throws IllegalArgumentException("Event not found: 99999")
→ GlobalExceptionHandler.handleBadRequest() catches it
→ HTTP 400 + renders error/error.html with 'Event not found: 99999'

DATA_ADDER accessing /admin/**:
→ @PreAuthorize throws AccessDeniedException
→ handleForbidden() → HTTP 403 + error page
```

13. WebClient & External API Call

service/AnthropicService.java · config/AppConfig.java

■ service/AnthropicService.java

Spring WebFlux's **WebClient** replaces the older RestTemplate for making HTTP calls to external APIs. It is configured once as a `@Bean` in `AppConfig` and then injected wherever needed.

```
@Service
@RequiredArgsConstructor
@Slf4j
public class AnthropicService {

    private final WebClient anthropicWebClient; // injected from AppConfig @Bean
    private final AnthropicProperties props; // injected from @ConfigurationProperties
    private final ObjectMapper objectMapper; // injected from AppConfig @Bean

    public Map<String, Object> chat(String userMessage, String context) {
        Map<String, Object> requestBody = Map.of(
            "model", props.getModel(), // "claude-opus-4-20250514"
            "max_tokens", props.getMaxTokens(), // 2048
            "system", buildSystemPrompt(context),
            "messages", List.of(Map.of("role", "user", "content", userMessage))
        );

        try {
            String responseJson = anthropicWebClient
                .post() // POST request
                .uri("/v1/messages") // https://api.anthropic.com/v1/messages
                .bodyValue(requestBody) // JSON body
                .retrieve()
                .bodyToMono(String.class) // response as String
                .block(); // wait synchronously

            JsonNode root = objectMapper.readTree(responseJson);
            String aiText = root.path("content").get(0).path("text").asText();
            // aiText = "{\"action\":\"CREATE\",\"message\":\"...\",\"data\":{\"...}}\"

            // strip markdown fences if AI wrapped response in ```json ... ```
            String cleaned = aiText.trim();
            if (cleaned.startsWith("```json")) cleaned = cleaned.substring(7);
            if (cleaned.endsWith("```")) cleaned = cleaned.substring(0, cleaned.length() - 3);

            return objectMapper.readValue(cleaned, Map.class); // JSON → Map

        } catch (WebClientResponseException e) {
            log.error("Anthropic API error: {} {}", e.getStatusCode(), e.getResponseBodyAsString());
            return Map.of("action", "ERROR", "message", "AI service unavailable.", "data", Map.of());
        }
    }
}

■ User types: "create a tech meetup next Friday at 6pm in Warsaw for 50 people"
→ POST to api.anthropic.com/v1/messages
→ AI returns: {"action":"CREATE","message":"Creating your event...",
"data":{"title":"Tech Meetup","date":"2025-04-18","time":"18:00",...}}
→ AiCommandService.handleCreate() → eventService.createEventFromMap()
```

■ Note

The **x-api-key** header is set once in the `WebClient @Bean` (from `AnthropicProperties.key`). Every call automatically includes it — you never handle authentication in `AnthropicService` itself.

14. Complete Annotation Quick Reference

All annotations used in the EventFlow project

Annotation	File / Layer	Effect on the Application
@SpringBootApplication	EventFlowApplication	Starts entire app: component scan + auto-config + config source
@ConfigurationPropertiesScan	EventFlowApplication	Auto-discovers @ConfigurationProperties classes
@EnableAsync	EventFlowApplication	Allows @Async methods to run in background threads
@ConfigurationProperties("pre fix")	EventFlowProperties, AnthropicProperties	Binds YAML keys under prefix to Java fields
@Data (Lombok)	All config/DTO classes	Generates getters, setters, equals, hashCode, toString
@Configuration	AppConfig, SecurityConfig	Class contains @Bean factory methods
@Bean	AppConfig	Registers WebClient and ObjectMapper as singleton beans
@RequiredArgsConstructor (Lombok)	Most classes	Constructor for all final fields — enables constructor injection
@Component	DataInitializer, SecurityUtils, LoggingAspect	Generic Spring-managed bean
@Slf4j (Lombok)	All services, AiCommandService, GlobalExceptionHandler	Injects 'log' field for SLF4J logging
CommandLineRunner	DataInitializer	run() called after context is ready — seeds initial data
@Transactional	All services — write methods	Wrap in DB transaction; rollback on RuntimeException
@Transactional(readOnly=true)	Service read methods	Faster reads — no dirty checking
@Entity	User, Event, Registration, AiConversation	JPA maps class to a database table
@Table(name=...)	User, Event, Registration, AiConversation	Sets exact DB table name
@Id	All entities	Marks primary key field
@GeneratedValue(IDENTITY)	All entities	DB assigns id via AUTO_INCREMENT on INSERT
@Column(...)	Entity fields	Customizes column: nullable, length, columnDefinition
@Enumerated(String)	Event.category/status, User.role, Registration.status	Stores enum as string ('ACTIVE'), not integer (0)
@ManyToOne(LAZY)	Registration.user/event, Event.createdBy	FK column in this table; loaded on access (lazy)
@OneToMany(mappedBy=..., ALL)	User.registrations, Event.registrations	Collection of children; cascade deletes

@JoinColumn(name=...)	Relationship fields	Specifies FK column name in the database
@CreationTimestamp	createdAt fields	Auto-set to current time on INSERT
@UpdateTimestamp	updatedAt fields	Auto-updated on every save()
@Getter/@Setter/@NoArgsConstructor (Lombok)	All entity classes	Generates boilerplate for JPA + Spring use
@ToString(exclude=...)	User, Event	Avoids infinite loop when lazy collections are in toString
@Repository	All repository interfaces	Spring Data bean; translates JPA exceptions
@Query("JPQL")	EventRepository, UserRepository, RegistrationRepository	Custom JPQL query when derived names are insufficient
@Param("name")	Repository @Query methods	Binds method parameter to :name in JPQL
@Service	All service classes	Business logic bean — one singleton per class
@Controller	All controller classes	MVC controller — methods return view names
@RequestMapping("/path")	All controllers (class level)	Base URL prefix for all methods in the class
@GetMapping("/path")	Controller methods	Handle HTTP GET at the combined path
@PostMapping("/path")	Controller methods	Handle HTTP POST at the combined path
@PathVariable	Controller parameters	Extracts {id} from URL path
@RequestParam(defaultValue=...)	Controller parameters	Reads ?name=value from query string
@ModelAttribute("dto")	Controller parameters	Binds HTML form fields to DTO
@Valid	Controller parameters	Activates Bean Validation on the annotated parameter
@ResponseBody	ManageController.chat()	Returns JSON directly instead of a view name
@RequestBody	ManageController.chat()	Deserializes JSON request body to Java object
@ResponseStatus(HttpStatus.X)	GlobalExceptionHandler	Sets HTTP status code on the response
@ControllerAdvice	GlobalExceptionHandler	Global handler — catches exceptions from all controllers
@ExceptionHandler(X.class)	GlobalExceptionHandler methods	Routes uncaught exceptions to this handler method
@EnableWebSecurity	SecurityConfig	Enables Spring Security's web filter chain
@EnableMethodSecurity	SecurityConfig	Enables @PreAuthorize on method level
@PreAuthorize("hasRole(...)")	AdminController, ManageController	Blocks access unless SpEL expression is true
@Aspect	LoggingAspect	Marks class as AOP aspect — defines cross-cutting behaviour
@Pointcut(...)	LoggingAspect	Defines named set of methods to intercept
@Around("pointcut")	LoggingAspect	Wraps matched methods — runs before and after
@UniqueConstraint	Registration @Table	DB-level UNIQUE(user_id, event_id) constraint

@NotBlank / @Email / @Size / @Min / @Max / @FutureOrPresent / @DecimalMin	DTO classes	Bean Validation rules — enforced by @Valid in controllers
--	-------------	--

15. How It All Flows Together

End-to-end request lifecycle in EventFlow

Example: A logged-in user registers for an event

```
// 1. Browser sends: POST /register-event/5

// 2. SecurityFilterChain checks: is user authenticated?
// @Bean SecurityFilterChain → .requestMatchers("/register-event/**").authenticated()
// → YES: user is logged in as john@example.com (ROLE_USER)

// 3. Spring MVC routes to PublicController
// @PostMapping("/register-event/{id}")
// @PathVariable Long id → id = 5

// 4. SecurityUtils.getCurrentUserOrThrow()
// reads SecurityContextHolder → finds "john@example.com"
// calls userRepository.findByEmail("john@example.com")
// returns User{id=3, role=USER}

// 5. registrationService.registerForEvent(user, 5)
// @Service → Spring calls the singleton RegistrationService
// @Around on LoggingAspect fires → starts timer
// @Transactional → Spring opens DB transaction

// checks:
// event.getStatus() == ACTIVE ✓
// event.isPast() == false ✓
// existsByUserAndEvent(user, event) == false ✓
// event.isFull() == false (30 spots, 12 registered) ✓

// new Registration(user, event) → status = CONFIRMED
// registrationRepository.save(reg) → INSERT into registrations
// @Transactional commits

// @Around completes → log.debug("registrationService.registerForEvent completed in 8ms")

// 6. return "redirect:/events/5?registered=true";
// Browser loads GET /events/5?registered=true
// Thymeleaf renders events/detail.html with a success banner
■ Console:
DEBUG → RegistrationService.registerForEvent completed in 8ms
INFO → User john@example.com registered for event 'Wrocław Jazz Night'

Browser:
→ Redirected to /events/5?registered=true
→ Page shows 'You are registered!' banner
→ Available spots: 187 (was 188)
```

Example: AI command from a DATA_ADDER

```
// User types in AI chat: "delete the jazz event"

// 1. POST /manage/ai/chat Body: {"message": "delete the jazz event"}
// @PreAuthorize("hasAnyRole('DATA_ADDER', 'ADMIN')") → passes (user is DATA_ADDER)
// @RequestBody AiChatRequestDto → request.getMessage() = "delete the jazz event"
// @ResponseBody → will return JSON, not a view

// 2. AiCommandService.handleMessage("delete the jazz event", user)
// @Transactional → opens transaction
// buildContext(user) → builds string of user's events with IDs and titles
```

```
// 3. anthropicService.chat(message, context)
// WebClient POST → https://api.anthropic.com/v1/messages
// Headers: x-api-key: sk-... (from AnthropicProperties @Bean)
// Body: {model, messages, system_prompt_with_context}
// AI responds: {"action":"DELETE","message":"I'll delete 'Wrocaw Jazz Night'", "data":{"title":"jazz"}}

// 4. Back in handleMessage → switch(action) → case "DELETE"
// handleDelete(data, user)
// finds Event with title containing "jazz" in user's own events
// eventService.deleteEvent(event.getId(), user)
// → LoggingAspect @Around fires
// → @Transactional nested
// → eventRepository.delete(event) → DELETE FROM events WHERE id=2
// → log.info("Event 'Wrocaw Jazz Night' deleted by manager@eventflow.com")

// 5. AiConversation saved to DB
// new AiConversation(user, "delete the jazz event", "I'll delete...", "DELETE")
// conversationRepository.save(convo)

// 6. @ResponseBody returns:
// {"action":"DELETE","message":"I'll delete 'Wrocaw Jazz Night'", "success":true,
// "data":{"deleted":true,"title":"Wrocaw Jazz Night"}}
■ Console:
DEBUG → EventService.deleteEvent completed in 14ms
INFO → Event 'Wrocaw Jazz Night' deleted by manager@eventflow.com

Browser (AI chat):
AI: "I'll delete 'Wrocaw Jazz Night'" + success badge
```

This guide covers every Spring annotation used in the EventFlow project. Each annotation plays a specific role — from wiring beans and managing database transactions to enforcing security and intercepting service calls with AOP.